DP Framework

1. Characterize structure of an opt. solution
2. Recursively define value of an opt. solution
3. Compute value of opt. solution bottom-up (fill table)
4. Construct opt. solution from computed information

Weighted Interval Scheduling

Input: set of requests $\{1\dots n\}$; i starts at $s(i)$, ends at $f(i)$, has weight $w(i)$. Output: max-weight subset of mutually compatible intervals.

- Sort by finish time: $f_1 \le \cdots \le f_n$. Define $p(j)$ for an interval j=largest index of $i<j$ s.t. i and j are disjoint.
- Either $j \in O_j$: $\mathrm{OPT}(j)=w(j)+\mathrm{OPT}(p(j))$; or $j \notin O_j$: $\mathrm{OPT}(j)=\mathrm{OPT}(j-1)$.

$\Rightarrow \mathrm{OPT}(j)$=opt. value on $\{1\dots j\}$; $\mathrm{OPT}(0)=0$.

$$\boxed{\mathrm{OPT}(j) = \max\big(w(j)+\mathrm{OPT}(p(j)),\ \mathrm{OPT}(j-1)\big)}$$

Memoized

```
M_Compute_Opt(j):
  if j=0: return 0; if M[j] not empty: return M[j]
  else: M[j]=max(w(j)+M_Compute_Opt(p(j)),M_Compute_Opt(j-1)); return M[j]
```

Cannot use bottom-up in recursion: may find at ancestor node that whole branch is not optimal.

Iterative bottom-up

```
  M[0]=0; for i=1 to n: M[i]=max(M[i-1],w(i)+M[p(i)])
```

Find solution at index: $j \in O_j$ *iff* $w(j)+\mathrm{OPT}(p(j)) \ge \mathrm{OPT}(j-1)$.

```
Find_Solution(j):
  if j>0:
    if w[j]+M[p(j)]>=M[j-1]: {output j; Find_Solution(p(j))}
    else: Find_Solution(j-1)
```

Video Game Problem

Stages $0\dots n$; start with E_0; lose e_i landing on stage i. Move costs: walk (+1)=50, jump-1 (+2)=150, jump-2 (+3)=350. Maximize energy at stage n.

$\Rightarrow \mathrm{OPT}(i)$=max energy at stage i. Recurrence formula:

$$\boxed{\mathrm{OPT}(i) = \max\begin{cases}\mathrm{OPT}(i-1)-50-e_i\\ \mathrm{OPT}(i-2)-150-e_i\\ \mathrm{OPT}(i-3)-350-e_i\end{cases}}$$

```
Video_Game(j):
  OPT(0)=E_0; OPT(1)=E_0-50-e_1; OPT(2)=max(E_0-150-e_2,OPT(1)-50-e_2)
  for i=3 to n: use recurrence formula
```

Subset Sum / 0-1 Knapsack

Capacity W units of time, have requests $\{1\dots n\}$ that can be scheduled between 0 to W each with time w_i. Maximize utilization.

- $\mathrm{OPT}(i)$ over $\{1\dots i\}$ fails: when $n \in O_n$, subproblem has capacity $W-w_n$, not W, so OPT are two different functions depending on case.

$\Rightarrow \mathrm{OPT}(i,w)$=value of opt. solution using subset of $\{1\dots i\}$ with max time w.

$$\boxed{\mathrm{OPT}(i,w) = \begin{cases}\mathrm{OPT}(i-1,w) & w<w_i\\ \max(\mathrm{OPT}(i-1,w),\ w_i+\mathrm{OPT}(i-1,w-w_i)) & \text{else}\end{cases}}$$

- Grid: $(n+1)\times(W+1)$ table; rows=items i, cols=capacity w.
- Base row $M[0,w]=0$ (no items, zero utilization). Fill row by row: $i-1$=one fewer item in grid; w vs $w-w_i$: skip or include item i ($w-w_i<w_i$ guards negative index). Each row propagate to $M[n][W]$ over all n items. Column 0=0 units of time, column 1=1 unit $\cdots$ W=full W units (actual answer).

Recurrence

```
Subset_Sum(n,w):
  array M[0][w]=0 for each w=0 to W
  for i=1 to n: for w=0 to W: apply recurrence to M[i][w]
  return M[n][W]
```

Bottom-Up

```
Subset_Sum(n,W):
  array M[0..n][0..W]; for w=0 to W: M[0][w]=0
  for i=1 to n: for w=0 to W:
      if w<w_i: M[i][w]=M[i-1][w]
      else: M[i][w]=max(M[i-1][w],w_i+M[i-1][w-w_i])
  return M[n][W]
```

0/1 Knapsack: Items have weight w_i value v_i. Replace $w_i \to v_i$ in max branch.

$\Rightarrow \mathrm{OPT}(i,w)$=max value using $\{1\dots i\}$, capacity w.

$$\boxed{\mathrm{OPT}(i,w) = \max\big(\mathrm{OPT}(i-1,w),\ v_i+\mathrm{OPT}(i-1,w-w_i)\big)}$$

- Runtime: $\theta(nW)$=pseudo-polynomial. Need $\log_2 W$ bits to represent W.
- Rewriting: $nW=n\cdot 2^{\log_2 W}$, exponential in input size $\Rightarrow$ NOT polynomial.

Polynomial: T polynomial to input size (number of items). Pseudopolynomial: T exponential to input's bit length.

Sequence Alignment

Given $X=x_1\cdots x_m$, $Y=y_1\cdots y_n$ (e.g. DNA), find minimum-cost alignment. A matching is a set of ordered pairs with the property that each item occurs at most once. An alignment is a matching with no crossing pairs: if $(i,j),(i',j') \in M$ with $i<i'$, then $j<j'$.

- SEASIDE vs. DISEASE: a perfect matching exists but pairs cross $\Rightarrow$ not an alignment, not similar.
- Cost: gap penalty δ per gap; mismatch cost α_{pq} with $\alpha_{pp}=0$. Similarity=min-cost alignment of X and Y.

$\Rightarrow \mathrm{OPT}(i,j)$=min-cost alignment of $x_1\cdots x_i$ and $y_1\cdots y_j$. Either $(x_m,y_n)\in M$ (match/mismatch); x_m unmatched (gap in Y); or y_n unmatched (gap in X).

$$\boxed{\mathrm{OPT}(i,j) = \min\begin{cases}\mathrm{OPT}(i-1,j-1)+\alpha_{x_iy_j}\\ \mathrm{OPT}(i-1,j)+\delta\\ \mathrm{OPT}(i,j-1)+\delta\end{cases}}$$

```
  OPT(i,0)=i*delta; OPT(0,j)=j*delta
  for i=1 to m: for j=1 to n: apply recurrence
```

- Runtime: $O(mn)$. Space: $O(mn)$.

Sequence Alignment (Memory-Efficient D&C)

Split X at midpoint into X_L and X_R. Key: find optimal split point k^* in Y.

- Forward DP with Y on rows, X_L on columns: init column 0 as $k\delta$ for row k (aligning none of X_L against $y_1\cdots y_k$ costs k gaps).
  - Define fwd as the last column as a 1D array.
  - Last column gives fwd[k]=optimal cost of aligning X_L with $y_1\cdots y_k$ $\forall\ 0\le k\le n$ (cumulative number of edits since first entry to k).
  - fwd[0]=$m_L\delta$ (X_L against none of Y); fwd[n]=opt. cost to align X_L $\forall\ Y$.
- Backward DP on (Y^{rev}, X_R^{rev}): init column 0 as $(n-k)\delta$ for row $(n-k)$ (aligning none of X_R against $y_{n-k}\cdots y_n$ costs $(n-k)$ gaps).
  - Last column gives bwd[k]=opt. cost of aligning X_R with $y_{n-k}\cdots y_n$.
  - bwd[n]=opt. cost of aligning X_R with all of Y.
- Optimal split: find k^* s.t. fwd[k^*]+bwd[$n-k^*$] is minimized over $0\le k\le n$. Recurse on $(X_L, Y[1..k^*])$ and $(X_R, Y[k^*+1..n])$.

```
MemEff(X,Y):
  m=|X|,n=|Y|
  if m<=2 or n<=2: return StandardDP(X,Y)
  mid=floor(m/2),X_L=X[1..mid],X_R=X[mid+1..m]
  fwd=LastCol(X_L,Y),bwd=LastCol(reverse(X_R),reverse(Y))
  best_k=0,best=fwd[0]+bwd[n]
  for k=1 to n:
    if fwd[k]+bwd[n-k]<best: best=fwd[k]+bwd[n-k],best_k=k
  return MemEff(X_L,Y[1..best_k]) + MemEff(X_R,Y[best_k+1..n])
StandardDP(X,Y):
  m=|X|,n=|Y|; for i=0 to m: OPT[i,0]=i*d; for j=0 to n: OPT[0,j]=j*d
  for i=1 to m: for j=1 to n:
    OPT[i,j]=min(a(x_i,y_j)+OPT[i-1,j-1],d+OPT[i-1,j],d+OPT[i,j-1])
  return OPT
```

Each D&C level does $O(mn)$ work total across all subproblems (halving each time: $mn+mn/2+\cdots\le 2mn$). Overall: Time $\Theta(mn)$, Space $O(m+n)$.

Matrix Chain Multiplication

Given n dense matrices $A_1\cdots A_n$, find multiplication order to min. operations.

- Key: last multiplication of $A_i\cdots A_j$ must involve the multiplication of results of $(A_i\cdots A_k)\cdot(A_{k+1}\cdots A_j)$ where $i\le k<j$.

$\Rightarrow \mathrm{OPT}(i,j)$=optimal cost for $A_i\cdots A_j$; $\mathrm{OPT}(i,i)=0$. R_i=rows of A_i; C_k=cols of A_k; C_j=cols of A_j. Time: $O(n^3)$.

$$\boxed{\mathrm{OPT}(i,j) = \min_{i\le k<j}\big\{\mathrm{OPT}(i,k)+\mathrm{OPT}(k+1,j)+R_iC_kC_j\big\}}$$

Bellman-Ford (Shortest Paths) {Using $G=(v,e)$ notation}

Directed graph, possibly negative weights. Negative cycle with path to t implies dist$=-\infty$. Find shortest s-t path.

- If no negative cycles: shortest path is simple (loop-free), at most $v-1$ edges.

$\Rightarrow \mathrm{OPT}(i,v)$=min cost of $u\to v$ path using $\le i$ edges. Goal: find $\mathrm{OPT}(n-1,s)$.

$$\boxed{\mathrm{OPT}(i,v) = \min\begin{cases}\mathrm{OPT}(i-1,v) & \le i \text{ edges}\\ \min_{w\in\mathrm{Adj}(v)}\big(\mathrm{OPT}(i-1,w)+c_{vw}\big) & \text{else}\end{cases}}$$

```
Bellman_Ford(G,s,t):
  n=number of nodes in G
  OPT(0,t)=0,OPT(0,v)=infty
  for i=1 to n-1: for v in V in any order: use recurrence formula
```

$\Rightarrow$Variant: visit nodes closer to t in number of edges to reduce passes.

	Bellman-Ford	Dijkstra
Time	$O(mn)$; $O(n^3)$ in dense graph	$O(m\log n)$
Neg. weight	yes	no

Negative cycle detection

- If $\mathrm{OPT}(n,\{e_v\})<\mathrm{OPT}(n-1,\{e_v\})$ for any $\{e_v\}$: negative cycle exists.
- Problem: cycle to t keeps ∞ distances (iteration 0), never detected.
- Fix: add super-sink t^* with 0-cost edges from every node v to t^*. Run B-F to t^*: for every iteration i [0, v], use i hops to find min path to t^*.

$\Rightarrow$With v nodes, B-F requires v iterations. If a u-v path still decrease from $v-1$-th to v-th itn., v is on negative cycle OR v has path to neg. cycle. In this case, DP goes toward t^*

Network Flow

A **flow network** is a directed graph $G=(V,E)$ with: (1) each edge e having non-negative int capacity c_e; (2) a single source $s\in V$; (3) a single sink $t\in V$.

- Assumptions: no edges enter s or leave t; at least one edge incident to each node ($e\ge v/2$ so $O(e+v)=O(e)$); all capacities are integers.
  - Capacity constraint: for each $e\in E$, $0\le f(e)\le c_e$.
  - Conservation of flow: $\forall\ v\in V\setminus\{s,t\}$, $\sum_{e\text{ into }v} f(e)=\sum_{e\text{ out of }v} f(e)$.
  - **Integrality**: if $\forall c(e)\in\mathbb{Z}$, there $\exists$ max flow that is integer-valued.

$\Rightarrow$Value of steady-state flow (no transient/burst): $v(f)=\sum_{e\text{ out of }s} f(e)$.

Max Flow Problem

Given flow network G, find s-t flow with max value. Greedy approaches fail. Create Residual graph G_f with same nodes as G. For each edge e in G:

- $f(e)=0$: G_f has 1 forward edge with cap. c_e (does not change if unused).
- $f(e)=c_e$: G_f has 1 backward edge with cap. c_e (reverse direction).
- $0<f(e)<c_e$: G_f has 1 fwd edge (cap. $c_e-f(e)$) and 1 bwd edge (cap. $f(e)$).

Lemma: if $\forall e\in E$, $c(e)\in\mathbb{Z}$, then F-F ends in $\le C$ itn where $C=\sum_{e\leftarrow s^*} c(e)$. $\because$ augmentation increases flow by at least 1, $\therefore$ max-flow $v(f)\le C$ guarantees termination iff all edges have non-negative integer capacities.

Augmenting Paths & Ford-Fulkerson

If P is a simple s-t path in G_f, bottleneck(P) is the min residual capacity of any edge on P. High-level strategy:

Find s-t path P in G_f; find bottleneck(P) for path; push flow along P equal to bottleneck; update G_f; repeat until no s-t path exists.

```
Aug(f,C,P):
  b=bottleneck(P) // P:=augmenting path found in G_res; f:=flow assignment
  for each edge (v,u) in P:
    if (v,u) is forward edge: increase f(e) by b
    else (v,u) is backward edge: let e=(u,v); decrease f(e) by b
  return f'
```

Ford-Fulkerson Algorithm (can solve cycles)

```
MaxFlow(G,s,t,C):
  init f(e)=0 for all e in G // C:=capacity func. used in bottleneck
  while there is s-t path in G_f:
    let P be simple s-t path in G_f; f'=Aug(f,C,P); f=f'; update G_f
  return f
```

Cuts & Max-Flow Min-Cut

Capacity of cut: $c(A,B)=\sum_{e \text{ out of } A} c_e$. Only count $A\to B$, not $B\to A$.

Let f be any s-t flow and (A,B) be any s-t cut, then $v(f)=f^{\text{out}}(A)-f^{\text{in}}(A)$.

⇒$v(f)\le f^{\text{out}}(A)\le \sum_{e \text{ out of } A} c_e=c(A,B)$.

- Weak duality: $v(f)\le c(A,B)$, so max flow is bounded by cap of $\forall$ s-t cuts.

Max-Flow Min-Cut Theorem: max value of an s-t flow (flow passing from $s\to t$) equals min capacity of an s-t cut (i.e., smallest total weight of edges which if removed would disconnect source from sink).

⇒Lemma: if there is no s-t path in G_{Ford}, then $\exists$ s-t cut $(A^*,B^*)\in G$ s.t. $v(f)=c(A^*,B^*)$ and has the min capacity of any s-t cut in G.

- Min cut is not always unique. E.g., $s\to\cdot\to\cdot\to t$.
- Check uniqueness: find min-cut closest to s and min-cut closest to t. If they are the same cut, min-cut must be unique.

Scaling Ford-Fulkerson

Idea: only augment along paths with "large" residual capacity.

```
Scaling_MaxFlow(G,s,t,C):
  delta=largest power of 2<=max cap in an edge
  while delta>=1:
    while there is s-t path in G_f(delta)>=delta:
      P=s-t path in G_f(delta); f'=Aug(f,C,P); f=f'; update G_f(delta)
    delta=next smallest power of 2 (e.g., 16->8)
  return f
```

Circulation feasible iff max flow=total demand=D.

Edmonds-Karp & Other Algorithms

Same as Ford-Fulkerson, but always use augmenting path that is the shortest path (BFS) with remaining capacity. Running time: $O(nm^2)$, strongly poly.

Max-flow algorithm comparison

⇒m: edges, n: vertices; flow network: BFS≈$O(m)$

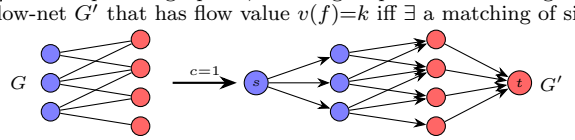
Algorithm	Time	Type
Ford-Fulkerson	$O(Cm)$	pseudo-poly
Scaling F-F	$O(m^2\log C)$	weakly poly
Edmonds-Karp	$O(nm^2)$	strongly poly

Bipartite Matching via Max Flow

Bipartite graph $G=(V,E)$: undirected graph that nodes can be partitioned as $V=X\cup Y$ s.t. every edge $e\in E$ connects one node in X and one in Y.

A **matching** M in G is a subset $M\subseteq E$ s.t. each node appears in at most one edge in M. **Max-size** match: add new edge will make a node appear twice.

Problem: Given bipartite graph G, find largest possible matching in G. Plan: design flow-net G' that has flow value $v(f)=k$ iff $\exists$ a matching of size k in G.



Solution

Find max flow f in G'. Edges with flow between (X,Y) correspond to max-size matching M in G. Prove G' has flow of value k iff G has matching of size k.

- If $\exists$ matching of size k in G: each independent s-t path gives c=1, so there is an s-t flow of $v(f)=k$ in G'.
- If flow $v(f)=k$ in G': must be k edges on cut (A,B) between (X,Y) that carry flow. These k edges cannot share any node → matching of size k in G.

Ford-Fulkerson is $O(Cm)$. Since all c_e=1, C=$O(n)$. T=$O(mn)$, strong poly.

Edge-Disjoint Paths

A set of paths is **edge-disjoint** if they do not share any edge. Problem: given directed $G=(V,E)$ and $s,t\in V$, find max number of edge-disjoint s-t paths.

⇒Design FN G' as G iff $\exists$ k edge-disjoint s-t paths in G. Set c_e=1 $\forall$ edges (already has s,t), remove t-s paths and do not count intermediate paths.

Solution

Find max flow f in G'. $v(f)$ equals max number of edge-disjoint s-t paths, so f identifies edges on these paths. Proof: show there are k edge-disjoint paths in G iff there exists a flow of value k in G'.

- If have k edge-disjoint paths in G, $\exists$ flow of value k in G' $\because$ each path c=1.
- Flow of value k in G': use conservation of flow to follow each unit from $s\to t$; remove all edges on that path from G', repeat k times to find k edge-disjoint s-t paths in G since there are k units of flow leaving s.

Handling flow cycles

Since there are ≥2 units of flow entering v, there must be ≥2 leaving. Remove cycle edges; use the other outgoing edge to continue toward t.

Flow cycles in undirected graphs

Replace each undirected edge with 2 directed edges in opposite directions (c=1 each). Problem: two paths will use $e=\{u,v\}$ in opposite directions.

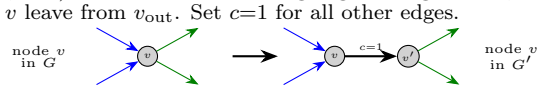
⇒Fix: before extracting s-t paths, remove all flow cycles on single edges.

Runtime with Ford-Fulkerson: $O(mn)$ (same as bipartite matching).

Node-Disjoint Paths

A set of paths is **node-disjoint** if paths between two nodes share no common node (except s and t). Given directed $G=(V,E)$ and $s,t\in V$, find max number of node-disjoint s-t paths.

⇒For each node $v\in V\setminus\{s,t\}$: split v into v_{in} and v_{out} connected by a single edge $(v_{\text{in}},v_{\text{out}})$ with c=1. All incoming edges of v go to v_{in}; all outgoing edges of v leave from v_{out}. Set c=1 for all other edges.



(General plan same as the previous problem.)

Circulation

A **circulation network** is a directed graph $G=(V,E)$ where each edge e has capacity $c_e\ge 0$ and each node $v\in V$ has a demand d_v.

- $d_v>0$: v is a sink (d_v flow leaves); $d_v<0$: v is a source ($|d_v|$ flow leaves).
- d_v=0: neither sink nor source.
- Note: a node that supplies flow has negative demand (source of flow).

Circulation $f(e)$ properties

- Capacity: for each $e\in E$, $0\le f(e)\le c_e$.
- Demand: $\forall$ node v, $\sum_{e \text{ into } v} f(e)-\sum_{e \text{ out of } v} f(e)=f^{\text{in}}(v)-f^{\text{out}}(v)=d_v$.
- If feasible circulation exists, then $\sum_v d_v=0$.
  Proof: $\sum_v d_v=\sum_v(f^{\text{in}}(v)-f^{\text{out}}(v))$. Each edge e contributes $+f(e)$ to one node's f^{in} and $-f(e)$ to another's f^{out}, so all terms cancel ($\sum_v d_v=0$).

Problem: Given a CN G with demands $\{d_v\}$, $\exists$ feasible circulation in G?

⇒Reduction to max-flow: make G' keep all edges/capacities from G. Add super-source s^* and super-sink t^*:

- For each node v with $d_v<0$ (source): add edge (s^*,v) with cap $|d_v|$.
- For each node v with $d_v>0$ (sink): add edge (v,t^*) with cap d_v.

Find max flow f in G' with total demand D.

- $v(f)<D$: no feasible circulation exists in G.
- $v(f)>D$: impossible (cuts closest to s^* and t^* each have capacity D).
- $v(f)=D$: feasible circulation exists; flow on original edges (i.e., strip edges from s^* and to t^*) gives valid circulation.

Reduction proof-of-correctness

1. If $\exists$ circ. f with demands $\{d_v\}$ in G, there is max flow of value D in G'. If G has circulation satisfying all demands, the net flow absorbed/produced by each demand node can be rerouted thru super-source/sink edges. Each node with $d_v>0$ was receiving d_v net flow, which now comes from (s^*,v), so all s^* edges are saturated. Thus max flow in G' achieves $D=\sum_{d_v>0} d_v$.
2. If $\exists$ max flow of value D in G', there is feasible circulation f with $\{d_v\}$ in G. If $v(f)$ in G'=D, every $e=(s^*,v)$ is saturated, i.e. demand is fully supplied. Remove s^*,t^* and redirect flow back to original demand gives $f\in G$.

Circulation with Lower Bounds

$G=(V,E)$ where edge e also has lower bound ℓ_e so $\ell_e\le f(e)\le c_e$. Demand, source, and sink are same. Problem: is there a feasible circ. in G?

⇒Reduce feasible circulation with lower bounds to feasible circulation.

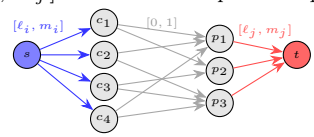
- Push initial flow f_0 where $f_0(e)=\ell_e$ through G to create flow imbalance L_v (find f_0 to satisfy all ℓ_e's), where total demand $D=\sum_{\text{pos}L_v}=|\sum_{\text{neg}L_v}|$: $L_v=\sum_{e \text{ into } v}\ell_e-\sum_{e \text{ out of } v}\ell_e$; disregard d_v in this step.
- Construct G' with $c'_e=c_e-\ell_e$, ℓ'_e=0, and $d'_v=d_v-L_v$.
- If asked, find feasible circ. f_1 in G' using max-flow reduction & remaining capacity. If none exists, no feasible circulation in G. Otherwise, $G=f_0+f_1$.

Survey Design Problem

Input: which customer bought which product; $\forall$ customer i, ask min/max questions (m_i,ℓ_i); $\forall$ product j, ask min/max questions (m_j,ℓ_j).

⇒Reduce to circulation with lower bounds.

- Build bipartite FN with source s, sink t, customer and product nodes:
  - $s\to i$: capacity $[\ell_i,\ m_i]$ ⋯ bounds on questions per customer.
  - $i\to j$ (i bought j): cap $[0,\ 1]$ ⋯ ask ≤1 ques. per product per customer.
  - $j\to t$: capacity $[\ell_j,\ m_j]$ ⋯ bounds on questions per product.



Why this is a valid reduction:

- Each unit of flow along path $s\to i\to j\to t$:=ask customer i one question about product j. Edge (i,j) exists only if i bought j, and cap $[0,1]$ ensures at most one question per customer-product pair.
- Lower/upper bounds on $s\to i$ edges force each customer to be asked between $[\ell_i,m_i]$ questions total (conservation: $f(e)$ into i=total questions asked of i)⇔bounds on $j\to t$ edges force each product to receive $[\ell_j,m_j]$ questions.
- A feasible circulation exists iff all bounds can be simultaneously satisfied; a feasible survey exists iff $v(f)=D$.
- Reduction: subtract lower bounds, construct G' with super-source s^* and super-sink t^*, and run max flow. A feasible survey exists iff $v(f)=D=\sum_{d_v>0} d_v$. If so, recover the assignment via $f_{uv}^{\text{original}}=f_{uv}^{\text{new}}+\ell_{uv}$.

Reduction Summary

Problem	Reduction	Runtime
Bipartite Matching	c_e=1, add s,t	$O(mn)$
Edge-Disjoint	c_e=1 on G	$O(mn)$
Node-Disjoint	node splitting, c=1	$O(mn)$
Circulation	add s^*,t^*	max-flow
Circ.+Lower Bds	2-pass, $c'=c-\ell$	max-flow